
FlashFormer: Whole-Model Kernels for Efficient Low-Batch Inference

Aniruddha Nrusimha¹ William Brandon¹ Jonathan Ragan-Kelley¹ Yoon Kim¹
Mayank Mishra² Yikang Shen² Rameswar Panda²

¹Massachusetts Institute of Technology ²MIT-IBM Watson AI Lab

{anin,wbrandon,jrk,yoonkim}@mit.edu

Abstract

The size and compute characteristics of modern large language models have led to an increased interest in developing specialized kernels tailored for training and inference. Existing kernels primarily optimize for compute utilization, targeting the large-batch training and inference settings. However, low-batch inference, where memory bandwidth and kernel launch overheads contribute are significant factors, remains important for many applications of interest such as in edge and latency-sensitive applications. This paper describes FLASHFORMER, a proof-of-concept kernel for accelerating single-batch inference for transformer-based large language models. Across various model sizes and quantizations settings, we observe nontrivial speedups compared to existing state-of-the-art inference kernels.

1 Introduction

As the utility and popularity of deep learning models has increased dramatically over the past decade and a half, there has been an increasing interest in low-level *kernel design*, i.e. developing specialized programs tailored to optimize particular workloads on the target hardware. This has been particularly true for transformer and transformer-like architectures. This is due to a dramatic increase in task performance across domains for these models as well as the increased *model scaling* (i.e. increasing parameter counts and computational costs per forward pass) and *inference scaling* (i.e. generating more tokens) needed to achieve these results.

While most accelerator vendors develop high performance kernels for common linear algebra routines such as matrix multiplication, these general-purpose kernels are often not sufficient for LLM inference. Consequently, there has been much recent work on kernels that accelerate the inference of transformer-based LLMs, such as FLASHDECODING [Dao et al., 2023] and PAGEDATTENTION [Kwon et al., 2023]. These kernels achieve efficiency gains by optimizing parts of operations common to the transformer inference pipeline. For example, FLASHDECODING modifies the work partitioning in FLASHATTENTION [Dao et al., 2022] for better parallelization during transformer decoding, while PAGEDATTENTION saves GPU memory by organizing the KV cache, thus allowing larger batch sizes.

Most kernels are optimized to maximize throughput for large matrix-matrix multiplications. The key to doing so on modern accelerators is to maximize the utilization of special matrix-multiply units (MMU)—called Tensor Cores on NVIDIA hardware and MXUs on TPUs—which provide most of the computational performance of the chip. In either large-batch inference or training, this design choice optimizes the primary bottleneck of these kernels, i.e. the computational cost of matrix multiplications with large arithmetic intensity i.e., computation per element of data movement).

Code: <https://github.com/dummy-link>

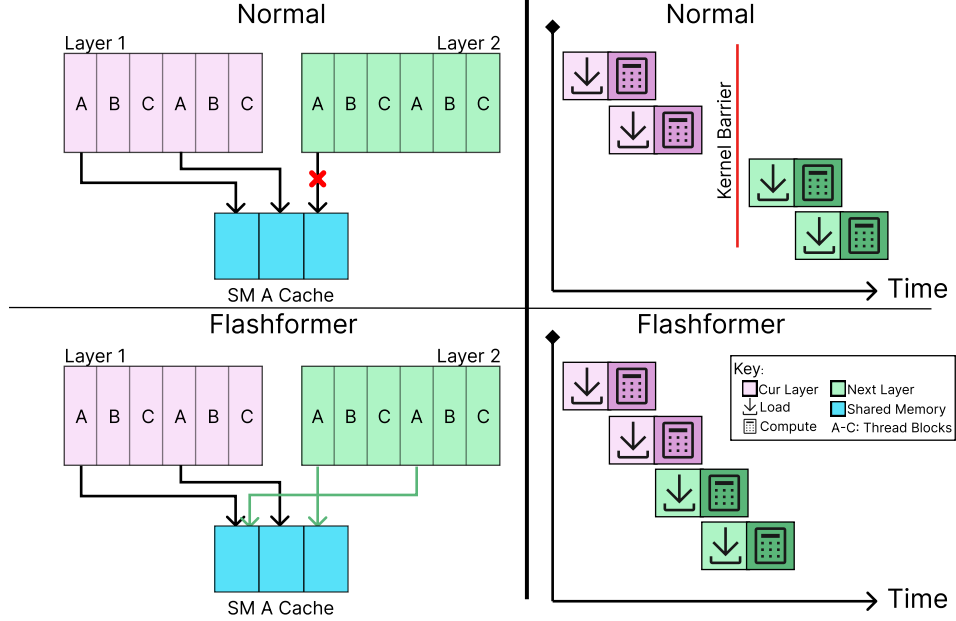


Figure 1: Depiction of the key benefit of FLASHFORMER. The top half depicts normal Transformer execution with multiple kernels. After all chunks of Layer 1 have been loaded into the SM cache, we cannot start loading Layer 2 (top left). This, coupled with kernel overhead, leads to a break in memory loads and computation (top right). With FLASHFORMER, we fuse kernels to enable overlapping across prior kernel barriers (bottom left). This leads to more efficient overlapping and faster runtime (bottom right).

However, many use cases operate under different constraints. For example, low-batch inference is primarily bottlenecked by the memory bandwidth costs of parameter and cache movement and depending on the size of the model, kernel launch overheads that arise from launching multiple separate kernels for each transformer layer. Low-batch inference workloads arise in many real-world applications, such as edge deployment, latency-sensitive applications, and more recently, inference-time scaling via longer rollouts. Moreover, low-batch inference does not obtain significant benefits from using MMUs on the hardware since the problem sizes are not bottlenecked by compute.

Writing fast kernels possible for these workloads requires further innovation along two common directions in kernel development research: 1) Increased operator fusion, or packing an increasing number of instructions and operations into a single kernel and 2) Increased specialization, or generating kernels that more closely optimized for the particular problem. This paper describes a strategy for writing a highly optimized inference kernel that runs the entire forward pass of a transformer-based large language model in a single kernel, and is specialized for a particular model architecture, hyperparameter configuration (such as the number of attention heads, embedding dimension etc), and the target hardware architecture. We use meta-programming, a shared pipelined buffer, and fast synchronization mechanisms to create a fully fused and specialized kernel. The resulting kernel, which we call FLASHFORMER, exploits increased overlapping of memory movement and the single fused kernel amortizes the launch overhead for the kernel over the entire forward pass. We apply this kernel to accelerate single-batch inference of Llama models, and find that it offers respectable speedups over state-of-the-art decoding kernels.

2 Background

2.1 Transformer Architecture

We target the Llama-3 [Grattafiori et al., 2024] family of models, which make use of the Transformer architecture with pre-RMSNorm [Jiang et al., 2023], Grouped Query Attention [Ainslie et al., 2023], RoPE [Su et al., 2021], and SwiGLU FFN layers [Shazeer, 2020]. Each layer of a Transformer model consists of six matrix operations, commonly grouped into five sublayers. Each sublayer has one

Matrix Operation	Matrix Load	Input	Output
QKV Projection	W_{QKV}	$\text{RMSNorm}(\mathbf{x})$	$Q[i], K[i], V[i]$
$QK^\top$ computation	$K[:, i]$	$Q[i]$	$S[i]$
PV computation	$V[:, i]$	$\text{softmax}(\alpha S[i])$	O
Attention Output Projection	$W_{a_{out}}$	$\text{vector}(O)$	a_{out}
GLU In/Gate Projection	W_{FFN1}	$\text{RMSNorm}(\mathbf{x} + a_{out})$	g_{inter}
GLU Output Projection	W_{FFN2}	$\text{SwiGLU}(g_{inter})$	g_{out}

Table 1: Llama matrix operations for decoding. Here i is the current sequence position, and $\mathbf{x}$ is the residual stream. W_{QKV} , $W_{a_{out}}$, W_{FFN1} and W_{FFN2} represent corresponding parameter matrices, and Q , K and V represent the query, key, and value matrices respectively. S represents the attention logits, P the attention weights, and a_{out} the output of attention. g_{inter} and g_{out} represent the output of the GLU input/gate projection and the final GLU output of the GLU layer respectively.

Model	Layers	d_{model}	d_{inter}	d_{head}	n_{q_heads}	n_{kv_heads}
Llama 3.1 8B	32	4096	14336	128	32	8
Llama 3.1 70B	80	8192	28672	128	64	8

Table 2: Configuration of Llama 3.1 8B and 70B, for which we develop FLASHFORMER kernel.

matrix operation, except for the Core Attention¹ layer, which has two. The matrix operations per layer with their respective input and output activations are listed in Table 1.

During autoregressive token-by-token generation, each matrix operation takes as input a large matrix stored in global memory as well as an activation computed by a previous operation. Transformer models are parameterized by their number of layers, model dimension, FFN dimension, head dimension, and the number of attention and key-value heads. The models we generate kernels for in this work are shown in Table 2.

2.2 GPU Programming and Architecture

Note that we use the NVIDIA GPU hardware terminology in this section, other architectures use different terms for similar hardware components. GPU programming optimizes over three interleaved hierarchies: the program hierarchy, the hardware hierarchy, and the memory hierarchy. The program hierarchy structures computations into kernels, thread blocks, warps, and threads to effectively expose parallelism. The hardware hierarchy maps thread blocks onto streaming multiprocessors (SMs) which is sub-divided into multiple warp schedulers responsible for running a warp which is a collection of 32 threads (this is configurable on some hardwares but not on NVIDIA GPUs) executing the same instruction. Individual threads are responsible for the core computation. Finally, the memory hierarchy manages the placement of data in registers, shared memory, and global memory to minimize latency and maximize throughput. Table 3 provides an overview of the three hierarchies.

At each level except the thread level, it is possible for multiple elements of the program hierarchy to exist on the same hardware if they satisfy certain hardware and memory constraints. In particular, multiple warps can exist on one warp scheduler provided there is enough space in registers, multiple thread blocks can colocate on an SM if there is enough shared memory and register space.

Program Level	Hardware Level	Count	Memory Pool	Capacity
Kernel	GPU	1	Global (HBM)	80 GB
Threadblock	SM	132	Shared	228KB/SM
Warp	Warp Scheduler	4/SM	-	-
Thread	Cuda Core	32/Warp	Registers	256KB/SM, 1KB/thread

Table 3: The hierarchies of the GPU and capacities on an H100 GPU. SM stands for Streaming Multiprocessor. Note that multiple thread blocks can coexist on an SM, and multiple warps on a warp scheduler.

¹Since “attention layer” could be ambiguous, we use “Core Attention” to refer to the operation $\text{softmax}(\alpha QK^\top)V$.

Python Implementation	Generated CUDA Kernel
<pre> def test_mul2(): @ch.kernel(ch.Params(n=ty.u32, x=ty. ptr_mut(ty.i32))) def kernel(n, x): i = ch.block_idx_x() * ch. block_dim_x() + ch. thread_idx_x() with ch.if_(i < n): x[i] *= 2 return ch.render(kernel) </pre>	<pre> __global__ void kernel(uint32_t n, int32_t* x) { uint32_t i = blockIdx.x * blockDim.x + threadIdx.x; if (i < n) { x[i] = x[i] * 2; } } </pre>

Table 4: Left: Python kernel source. Right: Generated CUDA kernel. Our metaprogramming language generates CUDA kernels from a Python interface. We use this to automate index math, perform static workload scheduling, and build kernel metaprogramming tools.

2.3 Decoding Workload and GPU Overlapping

The general strategy for overlapping in most GPU workloads is to partition the workload at each program level such that the corresponding hardware level is oversubscribed, i.e., the hardware has more work that it can execute. This oversubscription allows tasks to overlap: when one task is stalled awaiting resources—such as data loading from global to shared memory—other tasks continue executing arithmetic computations. Similarly, when one warp is waiting on a shared memory load, another warp can issue its computational instructions.

However, this strategy does not work well for inherently sequential workloads such as the sequential computations of Transformer layers. Each matrix operation in Table 1 depends on the completion of the operations before it, and each Transformer layer depends on the the completion of the previous Transformer layer. As a result, in normal execution there is no opportunity for overlapping between different kernels, as shown in Section 1. Furthermore, inference kernel launch overheads can cause a nontrivial slowdown for fast decoding kernels, as shown by Spector et al. [2023].

2.4 Operator Fusion

One solution to the problem of overlapping is kernel fusion, which involves taking two or more kernels and combining them into a single kernel. Existing kernel fusions can be divided into two types. The first type fuses a single (or multiple) pointwise operations (such as an activation function) into a matrix operation. These pointwise operations are directly applied to either the input of the matrix or its output or both. These types of fusions are now commonly automated through tools such as XLA [Sabne, 2020] or PyTorch Compile [Ansel et al., 2024].

The second type involves restructuring multiple operations to remove the need for global synchronization and reduce redundant data movement between multiple matrix operations. A notable recent example of this is the FLASHATTENTION series of kernels [Dao et al., 2022, Dao, 2024, Shah et al., 2024b], which fuses the QK^T computation, softmax calculation, and PV computation into one kernel reducing the memory movement from HBM to SRAM which significantly increases arithmetic intensity over a naive attention implementation.

3 Methods

3.1 General Kernel Design Principles

In this section, we elaborate on the key challenges in designing a family of whole-model kernels, and our implemented solutions.

Metaprogramming for Static Optimization: New kernels need to be implemented for each model tested. Writing such kernels manually is time intensive and prone to bugs. Implemented kernels greatly vary with both the dimensions of the models and quantization setups. To address this, we developed a metaprogramming language that, given the dimension sizes of the model, performs static workload partitioning based on predetermined rules to optimize performance. See Table 4 for an example.²

²While describing this metaprogramming language in full detail is outside the scope of this paper, we believe that this language will be of interest for kernel development more generally, and intend to open source it.

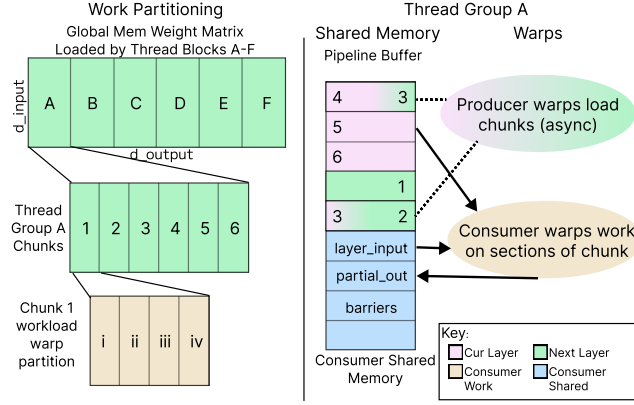


Figure 2: Left: Work partitioning between and within thread groups. The work for the matrix in global memory is split between thread groups. The work for each thread group is split into chunks for the pipeline buffer. Within the chunk, the work is divided into chunks. Right: Thread Group A work partitioning. Producer warps manage the pipeline buffer asynchronously, while consumer warps work synchronously on sections of chunks.

Because the exact workload is known at kernel generation, we can determine at compile time what all memory operations and computations are necessary. In the memory bound regime, we can load every element of our network parameters and KV cache exactly once. While this requires fully replicating the input, the cost of input data movement is much less than parameter and cache data movement. To efficiently generate these static workload schedules, we write a metaprogramming language in Python which automates the index math and static workload distribution and outputs CUDA kernels. As seen in fig. 2, the workload is divided among different thread blocks, then within a thread block into multiple chunks. Finally, each chunk is processed by a series of warps in a series of operations. Factors like inter-thread block workload partitioning, chunk sizes, and intra chunk workload partitioning are optimized to maximize utilization of hardware resources, especially global to shared memory bandwidth.

Memory pipeline: Each layer has different sized matrices and different shared memory requirements. As shown in fig. 2, we design and implement a unified memory pipeline that allows for the memory movement of arbitrary layers to overlap, and for memory movement to overlap all other operations. This overlapping of memory movement with computation is also referred to as a producer consumer pipeline design and is used by generally used by fast GEMM based kernels like FlashAttention-3 for Hopper GPUs [Shah et al., 2024a].

Specifically, each matrix listed in Table 1 is split into chunks of a fixed size in bytes. This chunk size remains the same throughout the model, although the data type of the chunk, number of elements, and matrix dimensions are subject to change. Each matrix’s chunks are split among thread blocks at compile time. While all weight matrix chunks are loaded every iteration, KV cache chunks are only loaded as needed. Due to this structure, different sections of the matrix buffer can store matrix chunks from different layers or sublayers. The number of matrix buffers stored at a given time is the “depth” of the pipeline, and the size of one matrix buffer is the size of each pipeline stage.

Global memory ordering: One problem with our approach is that the inputs for subsequent layers rely on the outputs of prior layer. Given the relaxed memory ordering of CUDA computations we use a per thread global memory fence coupled with an atomic based global synchronization mechanism to synchronize volatile reads and writes. This application of a traditional atomic synchronization ensures all participating threads’ global memory operations are visible to all other participating threads before continuing execution to avoid a read-after-write race condition. While atomic-based synchronization is far from novel, we believe we are the first to use it to combine different Transformer layers into a single fused kernel.

3.2 Transformer-specific Kernel Design

We now describe how the above principles are operationalized for the Transformer architecture.

Attention: We divide the attention sublayer into three parts, each of which has synchronous global communication between them: QKV Projection, Core Attention, and Attention Output Projection. A comparison of standard workload partitioning to ours can be found in fig. 3. A detailed diagram of the operations within each of these parts can be seen in fig. 5.

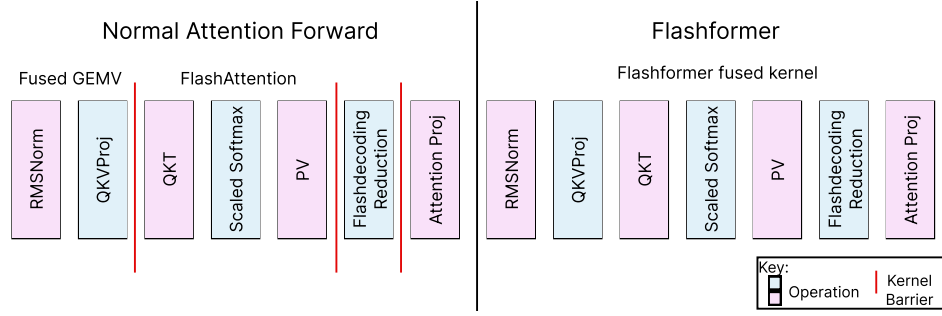


Figure 3: Conceptual of a standard attention forward pass and our method. Left: a standard attention forward pass in an inference pipeline, broken down into four kernels. Right: FlashFormer, where all operations are part of one kernel invocation. A detailed diagram of our attention setup can be found in fig. 5.

Both QKV Projection and Attention Output projection are matrix vector operations in the batch one case. For these operations, we cache the input in shared memory, and each chunk of the matrix corresponds to some number of rows of the weight matrix. We compute RMSNorm, RoPE, and then QKV projection, and add the outputs of the Attention Output projection directly to the residual stream.

For Core Attention, we use a modified version of the FLASHDECODING kernel [Dao et al., 2023]. Thread blocks are split among the different KV heads. For each KV head, each SM retains a replicated copy of queries which multiply with that particular KV head. The KV cache is then partitioned into matrix chunks, which are fed into the memory buffers. One problem with our pipelined approach is that we have no memory ordering that guarantees the elements of the KV cache correspond to the current token. We thus synchronously load the keys and values for the current token in.

After the entire KV cache has been read and the partial core attention output computed, we then perform a reduction per KV head before writing to a shared buffer. The reduction occurs in three stages. First, each thread block writes required metadata for the FlashDecoding reduction, e.g. the maximum logit and the softmax denominator. In the second stage, each thread block reads the combined metadata and computes the global softmax denominator. It then rescales its output by the ratio of the thread block’s softmax denominator to the combined softmax denominator, as in Milakov and Glimelshein [2018]. Finally, all threads add their rescaled outputs to a cumulative sum in global memory. As opposed to the normal FLASHDECODING kernel, the reduction is fused into the output computation kernel, removing one activation round trip.

GLU: In our kernel, the GLU in/gate operation is a matrix vector operation very similar to the QKV and Attention Output Projection operations in Attention, as shown in fig. 6. Unlike those operators, the output of each chunk of the GLU in/gate operation is not written to global memory.

Instead, we compute the SwiGLU activation on the outputs of the GLU in/gate function and then multiply it with a matrix chunk of GLU out. We then keep a vector of size d_{model} for accumulating partial outputs. With this approach, we can iterate through the entirety of the GLU computation without needing to do any synchronization or reduction. At the end the computation, we reduce all of these partial outputs along with the input.

4 Experiments & Results

4.1 Kernel Benchmark

We apply our kernel to three models: Llama 3.1 8B unquantized, Llama 3.1 8B quantized, and Llama 3.1 70B quantized. We compare with two strong baselines that are widely used in practice: GPTFast³ and vLLM⁴ [Kwon et al., 2023]. We enable all optimizations where possible. In particular, we enable CUDA graphs to decrease kernel launch overheads in our baselines.

We test a variety of prefill lengths, and generate 128 tokens per output. We measure only the speed of decoding. As a result, we only need to generate enough tokens to achieve a good estimate of the per-token latency with a given amount of prefill. All experiments are performed on one H100 SXM GPU, and use bfloat16 precision [Kalamkar et al., 2019] unless otherwise specified.

³<https://github.com/pytorch-labs/gpt-fast>

⁴<https://github.com/vllm-project/vllm>

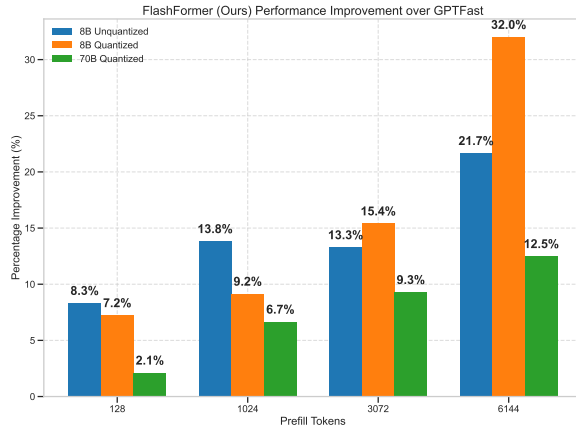


Figure 4: Percentage improvement in decode tokens per second of our kernel over GPTFast.

Unquantized Llama 3.1 8B: We first evaluate our kernel on an unquantized Llama 3.1 8B model. We find that our kernel achieves a 8-20% speedup over GPTFast, and up to a 61% speedup over vLLM at batch size 1. The results are in Table 5.

Prefill tokens (128 Output)	GPTFast	vLLM	FLASHFORMER (Ours)
128	168	146	182
1024	159	131	181
3072	158	123	179
6144	143	108	174

Table 5: Tokens Per Second (TPS) for Llama 3.1 8B decoding.

Quantized Llama 3.1 8B: We then compare a quantized version of Llama 3.1 8B. Our improvements over baselines in this arena are similar to the unquantized case, ranging from 7-32% as shown in Table 6. We quantize using GPTQ [Frantar et al., 2022] 4-bit weight only quantization in all cases, with a group size of 128. While this computation is likely bottlenecked by the operations needed for dequantization and not memory bandwidth, our approach allows us better to overlap dequantization with memory movement and computation.

Prefill tokens (128 Output)	GPTFast	vLLM	FLASHFORMER (Ours)
128	290	134	311
1024	284	156	310
3072	266	139	307
6144	228	124	301

Table 6: Tokens Per Second (TPS) for Llama 3.1 8b quantized (4-bit weight-only quantization) decoding.

Quantized Llama 3.1 70B: We finally evaluate on a large quantized model, Llama 3.1 70B. This model is also quantized to 4 bits with GPTQ. Our expectation was that the gains at 70B would be smaller than at 8B. This is because larger models have more opportunities for overlapping within a kernel. This decreases the need for overlapping between kernel launches. The results, shown in Table 7, are small improvements (2%) at short sequence lengths increasing to 13% at longer sequence lengths.

4.2 Ablation: Pipeline depth

We investigate how increasing pipeline stages affects kernel performance for the unquantized Llama 3.1 8B model. Our kernel design requires a minimum of two concurrent memory buffers during GLU computation. Consequently, a two-stage pipeline cannot overlap memory operations during GLU in/gate operations. The kernel was primarily optimized for a 64KB pipeline stage size. Performance measurements comparing two versus three pipeline stages at 3072 sequence length reveal that three stages achieves a 5% speedup over two stages, as shown in Table 8.

Prefill tokens (128 Output)	GPTFast	FLASHFORMER (Ours)
128	47	48
1024	45	48
3072	43	47
6144	40	45

Table 7: Tokens Per Second (TPS) for Llama 3.1 70b quantized decoding. vLLM is omitted due to crashes during quantization on our machine

Pipeline Stage Size	Number of Stages	TPS
64KB	2	170
64KB	3	179
32KB	3	135
32KB	4	138
32KB	5	130

Table 8: Llama 3.1 8B Unquantized pipeline depth ablation at sequence length 3072. We primarily optimize for 64KB pipeline stage size, and thus have worse workload partitioning at 32KB.

Due to shared memory capacity constraints, we cannot implement four pipeline stages at 64KB per stage. To test deeper pipelines, we halved the pipeline stage size, though these configurations were not as thoroughly optimized as the larger stage implementations. Results indicate that while four pipeline stages outperform three, five stages do not yield further improvements over four. At this depth, pipeline management overhead negates the benefits of deeper pipelining.

5 Discussion, & Future Work

5.1 Performance analysis

Two trends emerged in our experiments. First, FLASHFORMER had the largest speedups at the longest sequence lengths. We suspect this is due to our method of partitioning the KV cache. The size of our pipeline stages is 64KB, which corresponds to 256 keys or values with head dim 128 and bf16 precision. With 8 SMs per KV head, we need 4096 keys and values to provide one matrix chunk per participating SM. At longer sequence lengths this problem vanishes. Instead, FLASHFORMER overlaps more of the core attention computation and reduction with memory movement, avoiding inefficiency. Future work can explore better optimization of workloads at different sequence lengths.

The second trend we observe is that smaller models benefit more from our technique than larger models. This is likely due to the fact that very large models have more opportunities for overlapping within each operation, and thus gain less benefit from overlapping between operations.

5.2 Comparison to Theoretical Peak

Assuming our model is constrained by memory bandwidth, we can calculate the peak theoretical performance of our model, and what percentage of that performance we observe. If we compute bandwidth utilization as memory movement per token times throughput divided by bandwidth, we calculate that our kernel achieves roughly 82% bandwidth utilization, compared to the 68-75% of GPTFast.

It is unclear what portion of the memory bandwidth is achievable in realistic workloads. A linear layer achieves 86% utilization, while a pure copy kernel achieves 90% bandwidth utilization. As a result, we suspect that our model is starting to approach the practical hardware limits for batch size 1 generation on an H100.

5.3 Compile vs Runtime optimization

The work partitioning within our kernels—in particular, the partitioning of matrix chunks between thread blocks—is determined at compile time. This simplifies kernel logic and minimizes communication to global memory.

However, there are drawbacks to this strategy. At every point of synchronization, the entire GPU is held back by the slowest SM. Furthermore, it can lead to inefficiency when the amount of work

changes at runtime, which happens with the KV cache. For our workloads, we tried to avoid this problem through careful division of memory loads. Concurrent work like ThunderMLA [Spector et al., 2023] also implement a dynamic workload partitioning for a persistent FLASHATTENTION style Multi Latent Attention [DeepSeek-AI, 2024] kernel, controlled through a global instruction buffer. Their persistent kernel accesses a global memory buffer to determine what problem to work on next.

We hope that future work explores dynamic runtime optimized versions of FLASHFORMER, and explores the tradeoff between increased inter thread block communication to better balance the workload and decreased inter thread block communication to minimize per thread block latency.

5.4 Limitations

The single-batch setting we target is admittedly limited. We focus on this setting to demonstrate the practicality of the core ideas introduced in this paper, including aggressive specialization and the feasibility of a kernel design with a static workload schedule. We hope that future work applies these ideas to small batch and multi-GPU settings as well as to non-transformer models. Furthermore while we contribute a custom kernel metaprogramming library with customizable kernel templates, further work is needed to extend our abstractions to a greater variety of models and multi-batch / multi-GPU settings.

6 Related Work

Specialized Transformer model kernels. Due to the fact that the linear / GLU layers are standard linear algebra operations, most of the focus of specialized transformer kernel design has focused on optimizing attention. The FlashAttention [Dao et al., 2022, Dao, 2023, Shah et al., 2024b] restructures the attention computation to a numerically equivalent version that fused all of core attention. This general strategy is widespread and was used in many attention or attention-like operations.

Accelerator Kernel DSLs, such as Triton [Tillet et al., 2019], are used to simplify the kernel generation process by grouping computation into conceptual tiles. Specialized tools such as FlexAttention [Guesous et al., 2024] further allow for simple kernel generation for attention like workloads. Recently, automatic fusion of pointwise operators has continued to improve and become a staple of all modern machine learning libraries. XLA [Sabne, 2020] and PyTorch Compile [Ansel et al., 2024] automate fusion of pointwise operations for Jax [Bradbury et al., 2018] and PyTorch respectively.

Concurrent to this work, ThunderMLA [Spector et al., 2023] introduce a persistent kernel for MultiHead Latent Attention [DeepSeek-AI, 2024] layers that has each thread block dynamically schedule its own workload.

Low batch inference optimizations. Better kernel design is far from the only research area pursuing greater inference efficiencies. Two of the most important research directions in low batch inference are speculative decoding and quantization.

Speculative decoding [Leviathan et al., 2022] performs autoregressive inference on a small model and multi-token inference on a larger model. This relies on similarity between the model distributions and the relative speed of the small model. The critical idea was that the large verifier model would run at an effective batch size as long as the number of consecutive tokens fed into the model. Conditioned on the tokens verification, a single large model load could generate multiple tokens. Further iterations [Cai et al., 2024, Ankner et al., 2024, Li et al., 2024] improved upon this formula by using draft heads computed on the model’s outputs instead of a separate draft model.

Weight quantization methods [Frantar et al., 2022, Yao et al., 2022, Chee et al., 2023] have accelerated decoding by enabling lower and lower bit precisions for weight loads. This decreases the cost of memory movement, and thus improves inference speed. However, this is not perfect, as most of these methods utilize scarce GPU resources for dequantization.

7 Conclusion

We introduce a new method for overlapping operations between layers for transformer decoding workloads. We develop a family of kernels that leverage this new method of overlapping, and empirically verify the performance of these models on real workloads.

References

References

- Joshua Ainslie, James Lee-Thorp, Michiel De Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.
- Zachary Ankner, Rishab Parthasarathy, Aniruddha Nrusimha, Christopher Rinard, Jonathan Ragan-Kelley, and William Brandon. Hydra: Sequentially-dependent draft heads for medusa decoding, 2024.
- Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, CK Luk, Bert Maher, Yunjie Pan, Christian Puhersch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Michael Suo, Phil Tillet, Eikan Wang, Xiaodong Wang, William Wen, Shunting Zhang, Xu Zhao, Keren Zhou, Richard Zou, Ajit Mathews, Gregory Chanan, Peng Wu, and Soumith Chintala. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. ACM, April 2024. doi: 10.1145/3620665.3640366. URL <https://pytorch.org/assets/pytorch2-2.pdf>.
- James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Nécule, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/jax-ml/jax>.
- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning, ICMML'24*. JMLR.org, 2024.
- Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher M De Sa. Quip: 2-bit quantization of large language models with guarantees. *Advances in Neural Information Processing Systems*, 36: 4396–4429, 2023.
- Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
- Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. Flash-decoding for long-context inference, 2023. URL <https://crfm.stanford.edu/2023/10/12/flashdecoding.html>.
- DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model, 2024.
- Elias Frantar, Saleh Ashkboos, Torsten Hoeffer, and Dan Alistarh. GPTQ: Accurate post-training compression for generative pretrained transformers. *arXiv preprint arXiv:2210.17323*, 2022.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

- Driss Guessous, Yanbo Liang, Joy Dong, and Horace He. Flexattention: The flexibility of pytorch with the performance of flashattention, 8 2024. URL <https://pytorch.org/blog/flexattention/>. Accessed on March 29, 2025.
- Zixuan Jiang, Jiaqi Gu, Hanqing Zhu, and David Pan. Pre-rmsnorm and pre-crmsnorm transformers: equivalent and efficient pre-ln transformers. *Advances in Neural Information Processing Systems*, 36:45777–45793, 2023.
- Dhiraj Kalamkar, Dheevatsa Mudigere, Naveen Mellempudi, Dipankar Das, Kunal Banerjee, Sasikanth Avancha, Dharma Teja Vooturi, Nataraj Jammalamadaka, Jianyu Huang, Hector Yuen, Jiyan Yang, Jongsoo Park, Alexander Heinecke, Evangelos Georganas, Sudarshan Srinivasan, Abhisek Kundu, Misha Smelyanskiy, Bharat Kaul, and Pradeep Dubey. A study of bfloat16 for deep learning training, 2019. URL <https://arxiv.org/abs/1905.12322>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP ’23*, page 611–626, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702297. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding, 2022.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle: Speculative sampling requires rethinking feature uncertainty, 2024.
- Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax, 2018. URL <https://arxiv.org/abs/1805.02867>.
- Amit Sabne. Xla : Compiling machine learning for peak performance, 2020.
- Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision, 2024a. URL <https://arxiv.org/abs/2407.08608>.
- Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision. November 2024b.
- Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- Benjamin Spector, Aaryan Singhal, Dan Fu, and Chris Ré. Thundermla: Flashmla, faster and fused-er!, 2023. URL <https://hazyresearch.stanford.edu/blog/2025-03-04-thundermla>.
- Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding. *CoRR*, abs/2104.09864, 2021. arXiv: 2104.09864.
- Philippe Tillet, H. T. Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, MAPL 2019, page 10–19, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450367196. doi: 10.1145/3315508.3329973. URL <https://doi.org/10.1145/3315508.3329973>.
- Zhewei Yao, Reza Yazdani Aminabadi, Minjia Zhang, Xiaoxia Wu, Conglong Li, and Yuxiong He. Zeroquant: Efficient and affordable post-training quantization for large-scale transformers. *Advances in Neural Information Processing Systems*, 35:27168–27183, 2022.

A Detailed Algorithm Diagrams

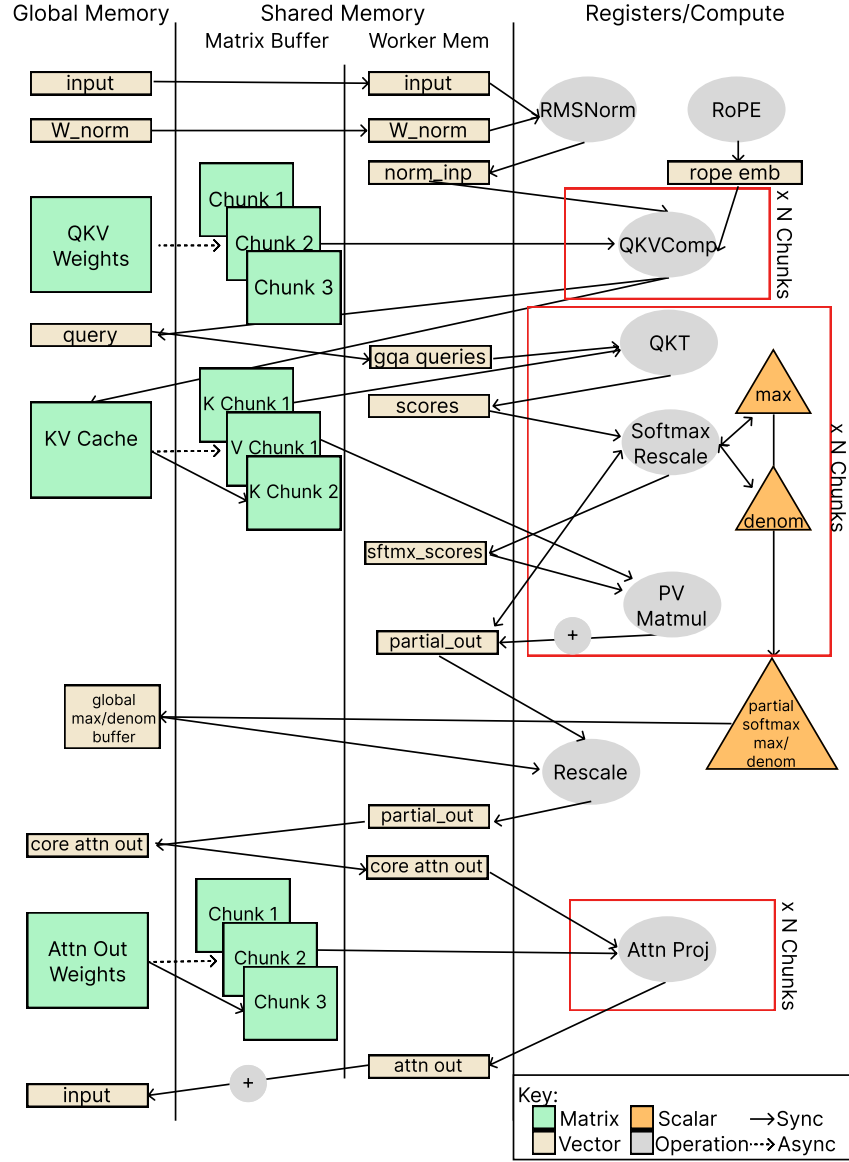


Figure 5: The Attention forward pass per layer. Large matrices (green) are loaded asynchronously from global, and do not wait on consumers. The consumers works synchronously, with 3 global synchronizations per attention forward pass. Each of the repeated computations (in red) corresponds to a asynchronously loaded matrix split into chunks.

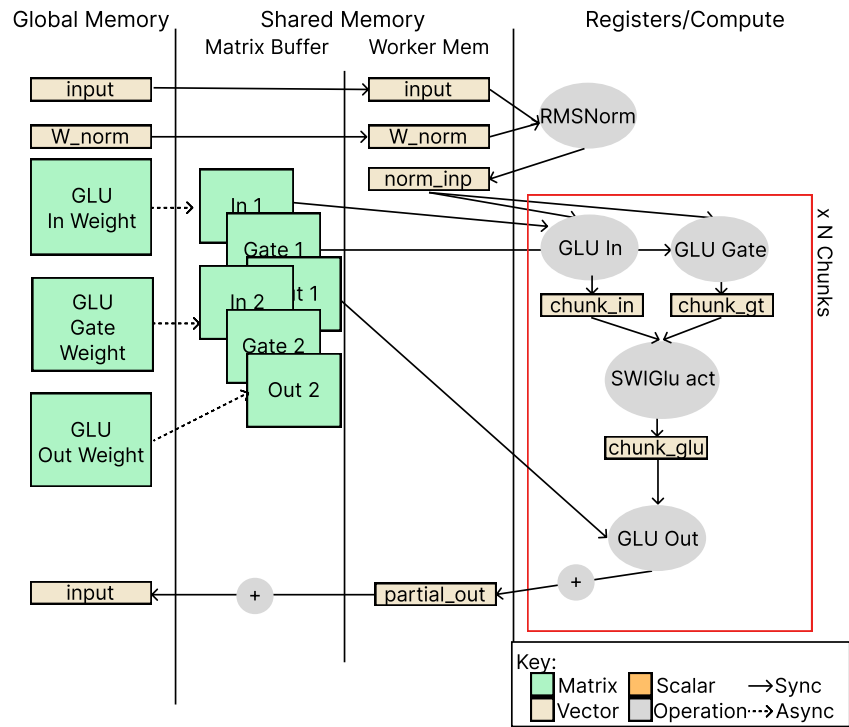


Figure 6: The GLU forward pass. Large matrices (green) are loaded asynchronously from global, and do not wait on consumers. The consumers works synchronously, with no synchronization to global within the sublayer.